

# ■ Binary Search

Algorithm Guide with Intuition & Visuals

11 Problems · From Basics to Advanced

| #  | Problem                              | Difficulty |
|----|--------------------------------------|------------|
| 1  | Search X in Sorted Array             | Easy       |
| 2  | Lower Bound                          | Easy       |
| 3  | Upper Bound                          | Easy       |
| 4  | Search Insert Position               | Easy       |
| 5  | Floor and Ceil in Sorted Array       | Easy       |
| 6  | First and Last Occurrence            | Easy       |
| 7  | Count Occurrences in Sorted Array    | Easy       |
| 8  | Search in Rotated Sorted Array       | Medium     |
| 9  | Find Minimum in Rotated Sorted Array | Easy       |
| 10 | How Many Times Array is Rotated      | Easy       |
| 11 | Single Element in Sorted Array       | Medium     |
| 12 | Find Peak Element                    | Medium     |

The Golden Template — use this for EVERY binary search:

```
lo, hi = 0, len(arr) - 1
while lo <= hi:
    mid = lo + (hi - lo) // 2 # avoids overflow
    if arr[mid] == target: return mid
    elif arr[mid] < target: lo = mid + 1
    else:                  hi = mid - 1
return -1
```

## 1. Search X in Sorted Array

■ EASY | LC 704

### Problem

Given a sorted array and a target X, return its index. Return -1 if not found.

### Visual

```
arr = [2, 4, 6, 8, 10, 12, 14]    target = 10

idx:  0   1   2   3   4   5   6
      [ 2,  4,  6,  8, 10, 12, 14 ]
      lo           ^           hi
                  mid
```

```
Step 1: lo=0, hi=6 → mid=3 arr[3]=8 < 10 → lo=4
Step 2: lo=4, hi=6 → mid=5 arr[5]=12 > 10 → hi=4
Step 3: lo=4, hi=4 → mid=4 arr[4]=10 == 10 → FOUND at index 4 ✓
```

### Key Intuitions

- Each step eliminates HALF the search space — this is why it's  $O(\log n)$ .
- $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) // 2$  prevents integer overflow (critical in C++/Java).
- While  $\text{lo} \leq \text{hi}$ : the  $=$  handles the single-element case correctly.
- If  $\text{arr}[\text{mid}] < \text{target} \rightarrow$  answer is to the RIGHT  $\rightarrow \text{lo} = \text{mid} + 1$
- If  $\text{arr}[\text{mid}] > \text{target} \rightarrow$  answer is to the LEFT  $\rightarrow \text{hi} = \text{mid} - 1$

### Algorithm (Python)

```
def search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 2. Lower Bound

■ EASY

### Problem

Find the index of the first position where  $\text{arr}[i] \geq \text{target}$ . (Smallest index where we can insert target keeping sorted order.)

### Visual

```
arr = [1, 3, 3, 5, 7, 7, 9]   target = 4

idx:  0   1   2   3   4   5   6
      [ 1,  3,  3,  5,  7,  7,  9 ]
                ^
        Lower Bound = index 3 (first element >= 4 is 5)

arr = [1, 3, 3, 5, 7]   target = 3
      [ 1,  3,  3,  5,  7 ]
                ^
        Lower Bound = index 1 (first 3)
```

### Key Intuitions

- Lower bound = first index where  $\text{arr}[i] \geq \text{target}$ .
- Use 'ans' variable — update it whenever  $\text{arr}[\text{mid}] \geq \text{target}$ , then go LEFT to find earlier.
- Different from exact search: we don't stop at  $=$ , we keep narrowing.

- If target > all elements, lower bound = n (one past end — valid insert position).
- This is the building block for Floor, Ceil, First Occurrence, Count problems.

### Algorithm (Python)

```
def lower_bound(arr, target):
    lo, hi = 0, len(arr) - 1
    ans = len(arr)      # default: target bigger than all
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] >= target:  # potential answer, try to go left
            ans = mid
            hi = mid - 1
        else:
            lo = mid + 1
    return ans
```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 3. Upper Bound

■ EASY

### Problem

Find the first index where  $\text{arr}[i] > \text{target}$ . (First element strictly greater than target.)

### Visual

```
arr = [1, 3, 3, 5, 7, 7, 9]   target = 3

idx:  0  1  2  3  4  5  6
      [ 1, 3, 3, 5, 7, 7, 9 ]
                ^

Upper Bound = index 3  (first element > 3 is 5 at index 3)
```

Lower Bound vs Upper Bound comparison:

```
target = 3
Lower Bound = 1  (first >= 3)
Upper Bound = 3  (first > 3)
Count of 3s = UB - LB = 3 - 1 = 2 ← very useful!
```

### Key Intuitions

- Upper bound = first index where  $\text{arr}[i] > \text{target}$ .
- Only change from Lower Bound: condition is  $\text{arr}[\text{mid}] > \text{target}$  (strict >).
- $\text{UB} - \text{LB}$  = count of occurrences of target in the array.
- UB points to the position AFTER the last occurrence of target.
- Together LB & UB define the 'equal range'  $[\text{lb}, \text{ub})$  — a closed-open interval.

### Algorithm (Python)

```
def upper_bound(arr, target):
    lo, hi = 0, len(arr) - 1
    ans = len(arr)      # default: target >= all elements
    while lo <= hi:
```

```

mid = lo + (hi - lo) // 2
if arr[mid] > target:    # strictly greater – potential answer
    ans = mid
    hi = mid - 1
else:                   # arr[mid] <= target, go right
    lo = mid + 1
return ans

```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 4. Search Insert Position

■ EASY | LC 35

### Problem

Given a sorted array and target, return index if found, else return the index where it would be inserted to keep the array sorted.

### Visual

```

arr = [1, 3, 5, 6]

target=5 → found at index 2 → return 2
target=2 → not found        → would go between 1 and 3
                                   → return 1
target=7 → not found        → would go after 6
                                   → return 4
target=0 → not found        → would go before 1
                                   → return 0

KEY INSIGHT: This is exactly Lower Bound!
Lower bound returns first index >= target = insert position

```

### Key Intuitions

- This problem IS the lower bound — exact same algorithm, different framing.
- After binary search exits, 'lo' points exactly to the insert position.
- lo is always the answer: it's where lo would be if we searched for  $\geq$  target.
- No need for a separate 'ans' variable — just return lo after the loop.

### Algorithm (Python)

```

def searchInsert(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return lo    # lo = correct insert position when not found

```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 5. Floor and Ceil in Sorted Array

■ EASY

### Problem

Floor = largest element  $\leq$  target. Ceil = smallest element  $\geq$  target. Return both.

### Visual

```
arr = [1, 2, 8, 10, 11, 12, 19]   target = 5

idx:  0   1   2   3   4   5   6
      [ 1,  2,  8, 10, 11, 12, 19 ]
          ^   ^
          floor ceil

Floor(5) = 2   (largest element  $\leq$  5)
Ceil(5)  = 8   (smallest element  $\geq$  5)

target = 10 (exact match):
Floor(10) = 10   Ceil(10) = 10

Floor = largest arr[mid]  $\leq$  target  $\rightarrow$  track in 'floor_ans'
Ceil  = smallest arr[mid]  $\geq$  target  $\rightarrow$  track in 'ceil_ans' (= lower bound)
```

### Key Intuitions

- Ceil = Lower Bound — they are identical.
- Floor: track last seen  $\text{arr}[\text{mid}] \leq \text{target}$ ; when  $\text{arr}[\text{mid}] \leq \text{target} \rightarrow$  update floor, go right.
- If target exists in array: Floor = Ceil = target.
- Floor doesn't exist if  $\text{target} < \text{arr}[0]$ . Ceil doesn't exist if  $\text{target} > \text{arr}[\text{n}-1]$ .
- Think of a number line: floor rounds DOWN, ceil rounds UP to nearest array element.

### Algorithm (Python)

```
def floor_ceil(arr, target):
    lo, hi = 0, len(arr) - 1
    floor_ans, ceil_ans = -1, -1

    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] <= target:      # potential floor
            floor_ans = arr[mid]
            lo = mid + 1            # try to find larger floor
        else:                      # arr[mid] > target – potential ceil
            ceil_ans = arr[mid]
            hi = mid - 1           # try to find smaller ceil

    # Handle exact match for ceil
    if floor_ans == target:
        ceil_ans = target

    return floor_ans, ceil_ans
```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 6. First and Last Occurrence

■ EASY | LC 34

### Problem

Given a sorted array (may have duplicates) and a target, find the first and last index of target. Return [-1, -1] if not found.

### Visual

```
arr = [5, 7, 7, 8, 8, 8, 10]   target = 8
```

```
idx:  0   1   2   3   4   5   6
```

```
      [ 5,  7,  7,  8,  8,  8, 10 ]
```

```
          ^
```

```
          ^
```

```
        first    last
```

```
Answer: [3, 5]
```

Finding FIRST occurrence (go LEFT on match):

```
mid=3 → arr[3]=8 == target → ans=3, hi=mid-1=2 (go left)
```

```
mid=1 → arr[1]=7 < target → lo=mid+1=2
```

```
mid=2 → arr[2]=7 < target → lo=mid+1=3 > hi → STOP → first=3
```

Finding LAST occurrence (go RIGHT on match):

```
mid=3 → arr[3]=8 == target → ans=3, lo=mid+1=4 (go right)
```

```
mid=5 → arr[5]=8 == target → ans=5, lo=mid+1=6
```

```
mid=6 → arr[6]=10 > target → hi=mid-1=5 < lo → STOP → last=5
```

### Key Intuitions

- Run TWO binary searches — one for first, one for last occurrence.
- First occurrence: on match, save answer and go LEFT (hi = mid-1) to find earlier.
- Last occurrence: on match, save answer and go RIGHT (lo = mid+1) to find later.
- First occurrence = Lower Bound when target exists.
- Last occurrence = Upper Bound - 1 when target exists.
- Always check if first != -1 before computing last to avoid edge cases.

### Algorithm (Python)

```
def first_last(arr, target):
    def find_first():
        lo, hi, ans = 0, len(arr)-1, -1
        while lo <= hi:
            mid = lo + (hi - lo) // 2
            if arr[mid] == target:
                ans = mid
                hi = mid - 1    # go LEFT to find earlier
            elif arr[mid] < target: lo = mid + 1
            else: hi = mid - 1
        return ans

    def find_last():
        lo, hi, ans = 0, len(arr)-1, -1
        while lo <= hi:
            mid = lo + (hi - lo) // 2
            if arr[mid] == target:
```

```

        ans = mid
        lo = mid + 1    # go RIGHT to find later
    elif arr[mid] < target: lo = mid + 1
    else: hi = mid - 1
    return ans

return [find_first(), find_last()]

```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 7. Count Occurrences in a Sorted Array

■ EASY

### Problem

Count how many times a target appears in a sorted array.

### Visual

```

arr = [2, 4, 4, 4, 4, 7, 9]    target = 4

idx:  0   1   2   3   4   5   6
      [ 2,  4,  4,  4,  4,  7,  9 ]
          ^               ^

    Lower Bound=1      Upper Bound=5

Count = Upper Bound - Lower Bound = 5 - 1 = 4  ✓

General formula:
count(target) = upper_bound(target) - lower_bound(target)

```

### Key Intuitions

- Don't scan linearly — use the LB & UB trick for  $O(\log n)$ .
- $\text{count} = \text{upper\_bound}(\text{target}) - \text{lower\_bound}(\text{target})$ .
- If  $\text{LB} == \text{UB}$ , target doesn't exist  $\rightarrow \text{count} = 0$ .
- This is equivalent to:  $\text{last\_occurrence} - \text{first\_occurrence} + 1$ .
- This pattern extends to: count elements in range  $[a, b] = \text{UB}(b) - \text{LB}(a)$ .

### Algorithm (Python)

```

def count_occurrences(arr, target):
    def lower_bound(t):
        lo, hi, ans = 0, len(arr)-1, len(arr)
        while lo <= hi:
            mid = lo + (hi-lo)//2
            if arr[mid] >= t: ans = mid; hi = mid-1
            else: lo = mid+1
        return ans

    def upper_bound(t):
        lo, hi, ans = 0, len(arr)-1, len(arr)
        while lo <= hi:
            mid = lo + (hi-lo)//2
            if arr[mid] > t: ans = mid; hi = mid-1
            else: lo = mid+1

```

```
        return ans

    return upper_bound(target) - lower_bound(target)
```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 8. Search in Rotated Sorted Array

■ MEDIUM | LC 33

### Problem

A sorted array was rotated at some pivot. Search for target. Return index or -1.

### Visual

```
Original: [1, 2, 3, 4, 5, 6, 7]
Rotated:  [4, 5, 6, 7, 1, 2, 3]   pivot at index 3

      LEFT half sorted   RIGHT half sorted
      [4, 5, 6, 7] | [1, 2, 3]
        lo      mid      mid+1  hi
```

Key observation: ONE of the two halves is ALWAYS sorted.

Search target=1:

```
lo=0 hi=6 mid=3 arr[mid]=7
Left half [4..7] is sorted. Is 1 in [4,7]? NO → search right
lo=4 hi=6 mid=5 arr[mid]=2
Right half [2..3] is sorted. Is 1 in [2,3]? NO → search left
lo=4 hi=4 mid=4 arr[mid]=1 == target → return 4 ✓
```

### Key Intuitions

- Critical insight: in a rotated array, at least ONE half is always sorted.
- Check which half is sorted: if  $arr[lo] \leq arr[mid]$  → left half is sorted.
- If target falls within the sorted half → search there, else search the other half.
- This maintains  $O(\log n)$  by always eliminating half the array.
- $arr[lo] \leq arr[mid]$  tells you left is sorted; otherwise right is sorted.

### Algorithm (Python)

```
def search_rotated(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            return mid

        # Check which half is sorted
        if arr[lo] <= arr[mid]:          # LEFT half is sorted
            if arr[lo] <= target < arr[mid]: # target in left half?
                hi = mid - 1
            else:                          # target in right half
                lo = mid + 1
        else:                             # RIGHT half is sorted
            if arr[mid] < target <= arr[hi]: # target in right half?
```



```

        lo = mid + 1
    else:
        # target in left half
        hi = mid - 1
return -1

```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 9. Find Minimum in Rotated Sorted Array

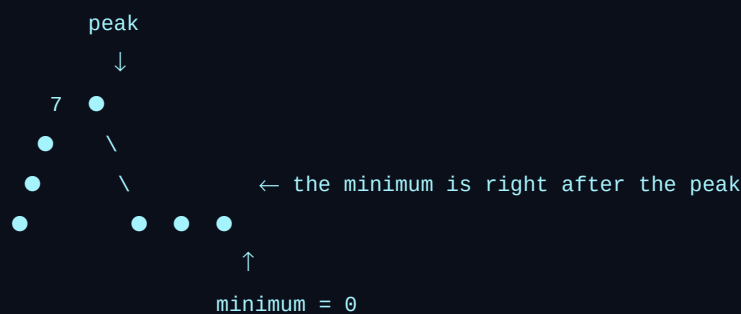
■ EASY | LC 153

### Problem

Find the minimum element in a rotated sorted array (no duplicates).

### Visual

Rotated: [4, 5, 6, 7, 0, 1, 2]



Step 1: lo=0 hi=6 mid=3 arr[mid]=7 > arr[hi]=2

→ minimum is in RIGHT half → lo = mid+1 = 4

Step 2: lo=4 hi=6 mid=5 arr[mid]=1 <= arr[hi]=2

→ minimum is in LEFT half (incl. mid) → hi = mid = 5

Step 3: lo=4 hi=5 mid=4 arr[mid]=0 <= arr[hi]=1

→ hi = mid = 4

lo==hi → minimum = arr[4] = 0 ✓

### Key Intuitions

- The minimum is the only element smaller than its predecessor (the inflection point).
- If  $\text{arr}[\text{mid}] > \text{arr}[\text{hi}] \rightarrow$  we're in the LEFT (larger) part  $\rightarrow$  minimum is to the RIGHT.
- If  $\text{arr}[\text{mid}] \leq \text{arr}[\text{hi}] \rightarrow$  mid could be the minimum or it's to the LEFT  $\rightarrow \text{hi} = \text{mid}$ .
- Use  $\text{hi} = \text{mid}$  (not  $\text{mid}-1$ ) because mid itself could be the answer.
- When  $\text{lo} == \text{hi}$ , we've found the minimum.

### Algorithm (Python)

```

def find_min(arr):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] > arr[hi]:
            # min is in right half
            lo = mid + 1
        else:
            # min is in left half (incl. mid)
            hi = mid
    return arr[lo]
# lo == hi == minimum index

```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 10. How Many Times is the Array Rotated?

■ EASY

### Problem

A sorted array was rotated  $k$  times. Find  $k$ . ( $k$  = index of the minimum element.)

### Visual

Original: [1, 2, 3, 4, 5, 6, 7] (k=0 rotations)

Rotate 1x: [7, 1, 2, 3, 4, 5, 6] min at index 1  $\rightarrow k=1$

Rotate 3x: [5, 6, 7, 1, 2, 3, 4] min at index 3  $\rightarrow k=3$

Rotate 4x: [4, 5, 6, 7, 1, 2, 3] min at index 4  $\rightarrow k=4$

Rotation count = Index of minimum element

Why? Each rotation moves the last element to the front.

After  $k$  rotations, the original first element (minimum) is at index  $k$ .

### Key Intuitions

- Rotation count = index of minimum element. These are identical problems.
- Reuse the Find Minimum algorithm — just return the INDEX, not the VALUE.
- If array is already sorted (not rotated)  $\rightarrow$  min is at index 0  $\rightarrow k=0$ .
- For  $n$ -element array:  $k$  rotations and  $(n-k)$  rotations give the same array.

### Algorithm (Python)

```
def count_rotations(arr):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] > arr[hi]: # min is in right half
            lo = mid + 1
        else: # min is in left half (incl. mid)
            hi = mid
    return lo # index of minimum = number of rotations
```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 11. Single Element in a Sorted Array

■ MEDIUM | LC 540

### Problem

Every element appears exactly twice except for one element. Find that single element in  $O(\log n)$ .

### Visual

arr = [1, 1, 2, 2, 3, 4, 4, 5, 5] single = 3

```
idx:  0  1  2  3  4  5  6  7  8
     [ 1, 1, 2, 2, 3, 4, 4, 5, 5 ]
               ^
           single (index 4)
```

KEY PATTERN before the single element:

Pairs are at (even, odd) indices: (0,1), (2,3) ...

$\text{arr}[0] == \text{arr}[1]$ ,  $\text{arr}[2] == \text{arr}[3]$  ...  $\leftarrow$  NORMAL

KEY PATTERN after the single element:

Pairs shift to (odd, even) indices: (5,6), (7,8) ...

$\text{arr}[5] == \text{arr}[6]$ ,  $\text{arr}[7] == \text{arr}[8]$  ...  $\leftarrow$  SHIFTED

At  $\text{mid}=4$  (even):  $\text{arr}[4]=3$ ,  $\text{arr}[5]=4 \rightarrow$  NOT equal  $\rightarrow$  single is here or LEFT

At  $\text{mid}=2$  (even):  $\text{arr}[2]=2$ ,  $\text{arr}[3]=2 \rightarrow$  equal  $\rightarrow$  single is to RIGHT

## Key Intuitions

- Before the single: pairs start at EVEN indices ( $\text{arr}[\text{even}] == \text{arr}[\text{odd}]$ ).
- After the single: pairs start at ODD indices ( $\text{arr}[\text{odd}] == \text{arr}[\text{even}]$ ).
- Always make mid even: if mid is odd, set  $\text{mid} = \text{mid} - 1$ .
- If  $\text{arr}[\text{mid}] == \text{arr}[\text{mid} + 1] \rightarrow$  we're in NORMAL zone  $\rightarrow$  single is to the RIGHT  $\rightarrow \text{lo} = \text{mid} + 2$ .
- If  $\text{arr}[\text{mid}] != \text{arr}[\text{mid} + 1] \rightarrow$  we're in SHIFTED zone or AT single  $\rightarrow \text{hi} = \text{mid}$ .

## Algorithm (Python)

```
def single_non_duplicate(arr):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if mid % 2 == 1:          # ensure mid is always even
            mid -= 1
        if arr[mid] == arr[mid + 1]: # pair intact  $\rightarrow$  single is to RIGHT
            lo = mid + 2
        else:                      # pair broken  $\rightarrow$  single is here or LEFT
            hi = mid
    return arr[lo]
```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## 12. Find Peak Element

■ MEDIUM | LC 162

### Problem

A peak element is one that is strictly greater than its neighbors. Find ANY peak index. Array may have multiple peaks.

Assume  $\text{arr}[-1] = \text{arr}[n] = -\text{infinity}$ .

### Visual

$\text{arr} = [1, 3, 5, 4, 2]$

```

    5  $\leftarrow$  PEAK
  /  \
 3    4
 /    \
1      2
```

Peak at index 2 ( $\text{arr}[2]=5 > \text{arr}[1]=3$  and  $\text{arr}[2]=5 > \text{arr}[3]=4$ )

Binary Search logic:

```

mid=2: arr[2]=5 > arr[3]=4 → we are on a DESCENDING slope
      → peak is to the LEFT (or at mid) → hi = mid
      (If arr[mid] < arr[mid+1] → ascending → peak to the RIGHT)

Ascending slope ■ → peak is somewhere to the RIGHT
Descending slope ■ → peak is HERE or to the LEFT

```

## Key Intuitions

- We don't need THE peak — any local peak is valid, so binary search works.
- If  $\text{arr}[\text{mid}] < \text{arr}[\text{mid}+1]$  → we're on an ascending slope → peak must be to the RIGHT.
- If  $\text{arr}[\text{mid}] > \text{arr}[\text{mid}+1]$  → we're on a descending slope → peak is here or to the LEFT.
- The boundaries act as -infinity, guaranteeing a peak always exists.
- This is a classic 'binary search without explicit target' — we search based on slope direction.
- When  $\text{lo} == \text{hi}$ , we've converged to a peak.

## Algorithm (Python)

```

def find_peak(arr):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] < arr[mid + 1]: # ascending → peak to the right
            lo = mid + 1
        else:                       # descending → peak here or to left
            hi = mid
    return lo # lo == hi == peak index

```

■ Time:  $O(\log n)$

■ Space:  $O(1)$

## ■ Quick Revision Cheatsheet

| Problem        | Core Trick                   | Key Line                                                   |
|----------------|------------------------------|------------------------------------------------------------|
| Search X       | Classic BS                   | <code>if arr[mid]==target: return mid</code>               |
| Lower Bound    | Track ans, go left on $\geq$ | <code>if arr[mid]&gt;=t: ans=mid; hi=mid-1</code>          |
| Upper Bound    | Track ans, go left on $>$    | <code>if arr[mid]&gt;t: ans=mid; hi=mid-1</code>           |
| Search Insert  | = Lower Bound                | <code>return lo (after loop)</code>                        |
| Floor          | Track on $\leq$ , go right   | <code>if arr[mid] &lt;= t: floor=arr[mid]; lo=mid+1</code> |
| Ceil           | = Lower Bound value          | Same as lower bound                                        |
| First Occur.   | On match go LEFT             | <code>ans=mid; hi=mid-1</code>                             |
| Last Occur.    | On match go RIGHT            | <code>ans=mid; lo=mid+1</code>                             |
| Count Occur.   | UB - LB                      | <code>return upper_bound(t) - lower_bound(t)</code>        |
| Rotated Search | Find sorted half first       | <code>if arr[lo] &lt;= arr[mid]: left sorted</code>        |
| Min in Rotated | Compare mid to hi            | <code>if arr[mid]&gt;arr[hi]: lo=mid+1</code>              |

|                |                        |                                    |
|----------------|------------------------|------------------------------------|
| Rotations      | = Index of minimum     | return lo (index, not value)       |
| Single Element | Even-index pair check  | if arr[mid]==arr[mid+1]: lo=mid+2  |
| Peak Element   | Follow ascending slope | if arr[mid] < arr[mid+1]: lo=mid+1 |

■ Remember: All these problems use the same  $O(\log n)$  binary search skeleton. The only difference is the condition inside and what you do on match.